

Final Report: Redirection Guard

ABSTRACT

This project introduces *Redirection Guard*, a Chrome extension that prevents redirect-based phishing attacks using metadata-driven detection. The system combines a dictionary-based lookup for known threats with a lightweight machine learning model to analyze URL metadata in real time. Legitimate and phishing samples are collected from trusted sources, then enriched with metadata and standardized through canonicalization and deduplication. This approach enables fast, privacy-preserving detection without locally rendering web content. The main challenge is the limited speed of data collection caused by API rate restrictions. Future development focuses on scaling dataset collection, training the model on cloud platforms, and integrating it into the extension for efficient and proactive phishing protection.

1. MOTIVATION

Phishing attacks remain one of the most persistent and damaging cybersecurity threats faced by internet users today. Redirect-based phishing represents a particularly interesting variant of this threat, where users begin their interaction on legitimate or seemingly safe websites before being silently redirected to malicious destinations. As Oest et al. note, phishing campaigns continue to evolve in sophistication, with a substantial portion of victim traffic occurring even after initial attack detection [1].

The success of these attacks stems from several compounding factors. Many users lack the technical knowledge to identify basic phishing attacks, let alone more sophisticated redirect-based attempts. Warning fatigue further compounds the problem, as constant alerts cause users to ignore legitimate warnings. These schemes are deliberately engineered by knowledgeable attackers who exploit human behavior and trust. Given these challenges, we formulate the following research question:

RQ: How can we design and implement a lightweight, real-time detection system that effectively identifies and prevents redirect-based phishing attacks at the browser level, while minimizing computational overhead, preserving user privacy, and reducing warning fatigue through intelligent threat discrimination?

This encompasses several critical sub-problems: Can URL metadata alone provide sufficient signal to distinguish malicious from benign redirects? What architectural design

enables real-time interception without introducing perceptible latency? How can we balance detection sensitivity with false positive rates to avoid warning fatigue?

We propose a Chrome browser extension that employs a two-tiered detection architecture. The first tier implements dictionary-based lookup for immediate blocking of known malicious domains. The second tier deploys a lightweight machine learning model that analyzes URL metadata when dictionary checks are inconclusive. This hybrid approach intervenes at the critical moment before redirect completion, preventing user exposure to malicious content entirely.

One key aspect of this approach is intervention at the point of vulnerability. Browser extensions can intercept requests as redirection occurs, enabling the system to flag and warn users before loading potentially malicious destinations. Our goal is to detect phishing websites without navigating to or rendering their content. Instead, we analyze the metadata of the redirecting URL request, including hostname, path, certificate details, and network attributes, to determine whether the site exhibits phishing characteristics.

By analyzing only URL metadata rather than rendering full page content, our system makes informed decisions before users are exposed to potentially harmful material. This approach enhances security, preserves computational efficiency, respects user privacy, and reduces ethical concerns associated with accessing potentially illegal content. The lightweight design ensures sustained user adoption by activating the AI agent only when dictionary checks are inconclusive, minimizing computational demands while maintaining comprehensive protection.

2. CHROME EXTENSION

2.1 Extension Architecture

The Redirect Guard Chrome extension implements a real-time phishing detection system using Chrome's Manifest V3 architecture. Built with TypeScript, the extension operates as an event-driven service worker that monitors all navigation events across the browser. The system consists of four primary components: a background service worker for navigation monitoring, a URL scanner for threat intelligence, an AI classification engine using Vertex AI's Gemini model, and a user interface system

comprising loading screens, warning pages, and a manual analysis popup.

A critical innovation in our extension is its ability to detect and intercept hidden automatic redirections that occur without user interaction. Traditional security solutions often miss these threats because they analyze pages only after they have loaded. Our extension addresses this gap by implementing comprehensive monitoring through Chrome's `webNavigation` API, specifically using the `onBeforeNavigate` and `onCommitted` event listeners [3]. The system detects three categories of redirects: server-side redirects (HTTP 3xx status codes), client-side redirects (JavaScript or meta refresh tags), and cross-domain navigation chains where the destination domain differs from the origin. This multi-layered detection is particularly effective against sophisticated phishing campaigns that employ redirect chains to obscure malicious destinations.

2.2 Interception and AI Integration

When a suspicious redirect is detected, the extension must analyze the URL without exposing the user to potentially malicious content. Since browser navigation cannot be truly "paused," our solution implements a navigation hijacking technique where the extension forcibly redirects the tab to an internal loading page before the suspicious destination loads. This effectively cancels the malicious redirect while displaying a user-friendly scanning interface. While the loading screen is visible, the background service worker queries VirusTotal's API to retrieve threat intelligence metadata, including detection counts, domain registration details, and SSL certificate information, all without rendering the malicious webpage.

The collected metadata is then sent to Google's Vertex AI platform for classification using the Gemini model, accessed through OAuth2 authentication via Chrome's `Identify` API [12]. The AI analyzes the structured metadata and returns a binary classification of "legit" or "phish." Based on this verdict, the extension either allows navigation to proceed (adding the URL to an in-memory allowlist for future visits) or blocks access and displays a detailed warning page. The warning page provides user override controls, allowing informed users to proceed if they believe the classification is incorrect, however; this feature is currently primarily used for testing purposes. This may be removed to prevent users from becoming accustomed to overriding all warnings, even if the site is truly malicious.

3. DATA COLLECTION

We collect and assemble a labeled dataset of URLs in three steps: (1) collect legitimate URLs, (2) collect phishing URLs, and (3) enrich each URL with detailed metadata using VirusTotal. This pipeline is designed to maximize realism, preserve safety (no direct visits to malicious sites), and produce a feature-rich dataset suitable for a metadata-only ML model.

3.1 Collect legitimate URLs

For legitimate data, we download the latest Tranco [2] ranking, which provides a list of the top one million most-visited domains compiled daily from multiple web traffic measurement sources. Due to our limited computational power, we randomly sample 5,000 urls from one million to represent active and reliable websites. This ensures that our "legitimate" category is both diverse and current, avoiding artificially constructed or inactive URLs while keeping the dataset manageable for enrichment and analysis.

3.2 Collect phishing URLs

For phishing URLs, our goal is to obtain real-world and actively updated samples rather than synthetic or outdated ones. We explored several url datasets. However, most of these were found to be unsuitable for our purpose: they are static, rarely updated, and primarily rely on natural language processing of URL strings for classification, making them unreliable for realistic model training. Hence, we adopted URLhaus [4] as the provider of the phishing domains.

URLhaus, maintained by Abuse.ch, provides another reputable and regularly updated dataset of malicious URLs. It is designed to share malware distribution and phishing links with researchers and cybersecurity organizations. URLhaus allows public downloads without special access requirements, offering us roughly 100,000 additional phishing URLs. These samples are tagged with timestamps, threat types, and status fields indicating whether each link is still active or offline.

By combining these sources, we ensure that our phishing dataset covers a broad spectrum of attack types, ranging from credential harvesters to redirect-based phishing schemes, but still remains representative of current global threat activity. Collectively, these feeds allow us to construct a comprehensive and realistic dataset for training a machine learning model focused on redirect detection.

3.3 Enrich each URL via VirusTotal

To avoid directly fetching potentially malicious content on our machines, we use VirusTotal [5] as an intermediary. Using their API, we built a scanner to uniformly extract the metadata we need without rendering or executing remote content locally. Below is our procedure to enrich our dataset:

1. We submit each URL in our dataset to the VirusTotal database via its API.
2. If a report already exists in VirusTotal’s database, we query its record (see *Features to extract* below) and append it to a new dataset.
3. If VirusTotal has no existing report for a URL, we add the URL to a local `waitlist.txt` and re-query later (VirusTotal requires some time to run the live scan and populate a report). We repeat this process until either a report appears or we reach a retry policy cutoff, ensuring we eventually obtain a complete feature set for each labeled URL.

However, the free VirusTotal plan enforces strict rate limits (e.g., 500 requests per day), hence, fully enriching a dataset of 25,000 URLs would take an extremely long time (up to 50 days of continuous querying even under ideal conditions). Our pipeline already includes exponential backoff, batching, and a persistent queue so that collection can safely continue across days, but the fundamental bottleneck remains the daily request cap.

3.4 VirusTotal API Restrictions

Unfortunately, we were unable to obtain access to VirusTotal’s premium API despite formally contacting them multiple times. We submitted several requests through their official channels, but after repeated attempts, we received no response. Because the premium API was unavailable and the free tier limits made the enrichment phase impossible to complete in a reasonable time, we had to implement workarounds.

Our solution was to create multiple free VirusTotal accounts, obtain API keys for each account, and design a batch extractor that automatically rotates through all keys. The scanner continues querying until every key reaches its daily limit, allowing us to significantly increase throughput. Because VirusTotal also tracks usage by IP address (not just by key) we additionally used NordVPN to rotate IPs and avoid global rate-limit blocks applied at the network level.

This workaround allowed us to complete the enrichment and fine-tune our model, but it also came with consequences: VirusTotal eventually detected the unusual pattern of API-key rotation and VPN-based IP changes, resulting in our primary IP being temporarily banned.

3.5 Ethical Considerations

Ethical and privacy considerations are central to our project design. Our dataset source, URLhaus [4], is a well-established and trusted platform widely used in academic and professional cybersecurity research. These providers ensure the authenticity and reliability of their threat intelligence data, making them suitable for academic use. While using data from these platforms, we strictly comply with their usage policies, licensing terms, and confidentiality requirements. By maintaining adherence to provider policies and institutional ethical standards, we ensure that our work remains both credible and compliant with established research norms.

All work conducted in this project was strictly for academic and research purposes. Our goal was to study phishing-URL detection methods, not to circumvent or exploit commercial API systems for personal gain. We initially attempted to obtain official Premium API access from VirusTotal through formal requests; however, after multiple unanswered attempts and given the strict limitations of the free tier, we implemented temporary workarounds solely to complete the dataset enrichment required for our research.

4. DATA ANALYSIS

4.1 Entropy and Similarity

To compress data and reduce the noise in the data, we incorporate two lightweight string-based features: Shannon Entropy and Levenshtein Similarity. Shannon Entropy measures the randomness of characters in a string, allowing the system to detect obfuscated or machine-generated strings commonly used in phishing attacks [12].

In parallel, Levenshtein Similarity quantifies how closely two strings resemble each other, enabling the system to identify embedded URLs or page titles that imitate legitimate domains while differing by only a few character edits [13]. Together, these features provide fast and effective first-line indicators of suspicious URL behavior.

4.2 Primary Features

From each VirusTotal [5] report, we extract a consistent set of metadata fields including, but not limited to, the following primary features:

Metadata	Detailed Features
URL Info	Entropy, Hostname
Content	Title, LogoURL, MIMEType, Services, Features,
Reputation	Partner votes, historical detections.
Redirection	Count, Entropy, Similarity
Domain	DomainAge, DomainValidDays
Network	Owner, Country
TLS	SubjectMatch, ValidDays, SANCount
EmbeddedURL	Count, Entropy, Similarity
Protocol	SecurePolicy, TransportSecurity, xFrameOption, CacheControlPolicy

Table 1. Primary Metadata Features Extracted from VirusTotal API

4.3 Data Pre-processing

To maintain the reliability and accuracy of our dataset, we implement several data hygiene measures throughout the collection and enrichment process.

The first step involves cleaning and deduplication at the source level. We process the phishing dataset by skipping the first 8 lines of metadata headers and normalizing all column names by stripping whitespace, removing '#' prefixes, and converting to lowercase. For the legitimate URL dataset from Tranco, which lacks a standard header, we extract URLs from column B and prepend "https://" to ensure proper protocol specification across all legitimate entries. Both datasets undergo deduplication based on exact URL matches, and any blank or empty URL entries are removed before sampling.

Our dataset preparation employs a controlled sampling methodology to create balanced or custom-ratio datasets. The `csv_process_mixed.py` script accepts user input to specify the desired number of phishing and legitimate URLs, validates that sufficient data exists in the source files, and performs random sampling using a configurable seed

value (default seed=42) to ensure reproducibility. The sampled URLs from both classes are then combined and shuffled randomly, with each entry assigned a sequential order number. This approach allows for flexible experimentation with different dataset compositions, class ratios, and randomization seeds while maintaining the ability to reproduce specific configurations.

To preserve traceability and enable future auditing, we maintain three parallel output files. The main mixed dataset contains order numbers, URLs, and binary labels ("phish" or "legit"). A phishing relation file links each phishing URL back to its complete original metadata from URLhaus. Finally, a legitimate URLs file tracks the order assignments for benign entries.

Quality assurance is enforced through automated duplicate checking via `csv_dupe_checker.py`, which scans the final mixed dataset to identify any duplicate URLs that may have inadvertently appeared despite source-level deduplication. The script reports total rows, unique URLs, and generates detailed reports of any duplicates found, including their assigned order numbers and labels. This verification step is critical for preventing data leakage, where the same URL appearing with different labels could compromise model training by allowing the model to memorize specific instances rather than learning generalizable features.

For storage, we use newline-delimited JSON (NDJSON) format during the data collection phase because it enables efficient streaming and incremental writing as new results are returned from the VirusTotal API. Another reason for choosing JSON is that our dataset contains several array-type fields, such as redirect chains, vendor labels, and embedded URLs, which are hard to represent in CSV files. JSON naturally supports hierarchical and array-based data structures, allowing us to store these complex fields without loss of information or formatting issues. Since modern data analysis libraries such as Pandas can natively read and process JSON files, we do not need to convert the dataset into another format for model training. This approach simplifies our workflow while preserving the flexibility and completeness of the collected data.

The finalized dataset consists of 15,666 URL samples, which were partitioned into three stratified subsets to support model development and evaluation. Specifically, 85% (13,316 samples) were allocated for training, 10% (1,566 samples) for validation, and 5% (784 samples) for testing. Each subset preserves the overall class distribution of the corpus, maintaining a consistent ratio of 60% phishing and 40% legitimate URLs. This stratification

ensures that model performance is evaluated under class proportions representative of the full dataset.

5. RESULTS

5.1 Model Configurations

We selected three models from Google: Gemini-2.5-pro, Gemini-2.5-flash, and Gemini-2.5-lite to tune and evaluate performance trade-offs across capability, speed, and cost. They are offered in Google Vertex AI, which provides the computational resources necessary for large-scale experimentation while offering cost-effective access through its introductory credit program. All models were trained under consistent hyperparameters, including 5 epochs, a 0.5 learning-rate multiplier, and adapter sizes of 8/8/16, alternatively, enabling a controlled comparison of how each model architecture responds to identical training conditions.

5.2 Training Comparison

The Gemini-2.5-Lite model exhibits fast and stable convergence, as shown in Fig. 1 and Fig. 2. The accuracy curve rises sharply during the early training steps and surpasses 90% within the initial phase before stabilizing near 99-100% for the remainder of training. The loss curve follows a consistent trend, dropping from an initial value near 2.0 to close to zero early in training and staying near zero with only minor, isolated spikes caused by harder samples. Overall, the Gemini-2.5-Lite model demonstrates that the Lite model efficiently learns the task, achieving very low training loss and maintaining high accuracy throughout training despite its smaller capacity.

The training dynamics of the Gemini-2.5-Flash model also demonstrate fast and stable convergence, as shown in Fig. 3 and Fig. 4. The accuracy curve rises sharply during the early training steps, exceeding 90% quickly and stabilizing near 99-100% for the remainder of training with minimal fluctuations. The loss curve follows a similar pattern, dropping from an initial value of roughly 1.8 to near zero early in training and remaining close to zero thereafter, aside from occasional small spikes associated with harder samples. In general, the curves indicate that the Flash model also learns efficiently, reaches low training loss rapidly, and maintains consistently high accuracy throughout training.

The learning curves of the Gemini-2.5-Pro model show rapid and stable convergence, as illustrated in Fig. 5 and Fig. 6. Similarly, its accuracy increases sharply in the early steps, reaching above 90% quickly and stabilizing near

99-100% for the remainder of training, with only minor expected fluctuations. The loss curve similarly drops from about 0.22 to near zero early on and remains close to zero with occasional small spikes from harder samples. It is safe to conclude that the Pro model also has efficient learning, low training loss, and consistently high accuracy throughout training.

Notably, all three models, Gemini-2.5-Lite, Gemini-2.5-Flash, and Gemini-2.5-Pro, exhibited unnoticeable training time and prompt-to-response latency, despite their differences in size and capability. As shown in Fig. 1–3, each model surpasses 90% accuracy early in training and converges to approximately 99-100% accuracy with near-zero loss, differing only in the magnitude of initial fluctuations. While the Lite model shows slightly more variability in the early steps, all three models ultimately achieve similarly stable convergence. These results indicate that, for this task and dataset scale, model capacity does not significantly impact computational cost, and all variants learn the classification task with comparable efficiency.

5.2 Final Evaluation

The Fig. 1 below shows the confusion matrix and metrics of the Pro model, but other models also achieve 100% accuracy on the evaluation split. However, this result should be interpreted with caution.

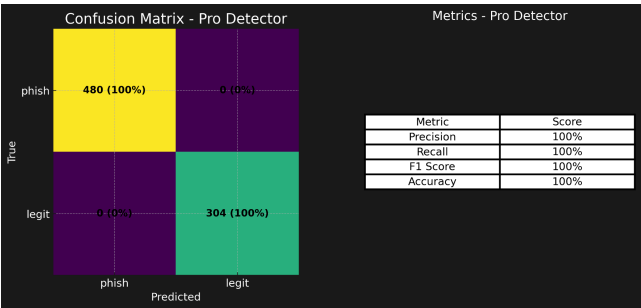


Figure 7. Final Evaluation of Gemini-2.5-Pro Model

First, the evaluation set represents only 5% of the total dataset, which may not fully capture the variability of real-world phishing URLs; an imbalanced or insufficient test set can artificially inflate performance metrics.

Second, such perfect scores may indicate overfitting, especially given that modern LLMs are extremely powerful relative to the modest dataset size of approximately 15k samples, making it plausible that the models memorized patterns rather than learned generalizable behavior.

Third, the dataset itself contains noise originating from VirusTotal partner votes, which act as extremely strong labels. These signals can encourage models to rely on dictionary-like memorization instead of true predictive reasoning. This issue becomes evident when evaluating newly reported URLs: the model sometimes classifies them as *legitimate* initially, but once VirusTotal partners later update their votes, the same URL is then classified as *phishing*. This discrepancy highlights that the detector is partially memorizing the labeling artifacts of the dataset rather than the intrinsic features of phishing behavior.

6. CHALLENGES & FUTURE PLAN

We have come a long way with our project over the past month. We came across issues that hampered our model's fine tuning time, but we managed to meet the goal on time. However, there is still one glaring challenge we acknowledge we could have been able to resolve.

Originally, to fine tune our model like we had envisioned, we had thought of a solution which involved upgrading our free API to the premium tier of the API provided by VirusTotal. We had contacted the VirusTotal team and had even mentioned Professor Kim's as our supervisor to demonstrate the academic goal of our project. A few days passed, and we received no response from the VirusTotal team, so we assumed we were not given access to the premium API and continued using the *workarounds* using the free tier (see *VirusTotal API Restrictions* section under *Data Collection* above) to accumulate data extraction.

7. RELATED WORK

The Redirect Guard builds upon existing phishing detection research while addressing gaps in redirect-based attack prevention. Jabir et al. provide a systematic review of phishing attacks, emphasizing user education rather than automated defenses [6]. Wang presents a LLaMA-based system for social media phishing, but its computational demands make it impractical for real-time browser deployment [7]. Our approach prioritizes efficient detection suitable for client-side execution and targets redirect-based attacks specifically.

WebSecAsst is most comparable, implementing a Chrome extension using XGBoost for malicious website classification [8]. However, it focuses on malicious file downloads rather than redirect interception. Our system analyzes URLs at the moment of redirection using metadata-only detection and employs a two-tiered architecture that balances detection coverage with computational efficiency.

Darktrace represents a commercial approach using self-learning AI to detect behavioral anomalies across organizational networks [9]. PIXM Phishing Protection employs AI-powered computer vision to visually analyze websites and detect brand impersonation [10]. While both demonstrate AI effectiveness for phishing detection, Darktrace requires company-wide adoption and deployment using substantial resources. Meanwhile, PIXM necessitates page rendering and server-side processing, introducing latency and privacy concerns. Our browser extension targets individual users with client-side metadata analysis that detects threats before content loads. These works establish foundations for AI-assisted phishing detection but do not address lightweight, real-time redirect interception with privacy-preserving metadata analysis at the browser level.

8. FINAL BUDGET

Our project leverages Google Cloud's Education Program, which provides students with up to \$300 in free credits for Vertex AI and other Google Cloud services [11]. This approach eliminated out-of-pocket expenses while providing access to professional-grade machine learning infrastructure. We utilized Google Cloud's Vertex AI Studio for model development, which offers access to over 200 foundation models through Model Garden and supports efficient model tuning with our collected phishing dataset. The platform's managed infrastructure eliminates the complexity of manual GPU configuration, allowing us to focus on model development and Chrome extension integration rather than infrastructure management.

However, our experience with Vertex AI training encountered significant complications. Although Google advertises \$300 in free credits for new accounts, the actual billing behavior proved unpredictable. Training several Lite and Flash variants initially appeared manageable, costing approximately \$50 in total. In contrast, when we initiated training for the Pro model, the system immediately escalated the projected charges to over \$700, ultimately resulting in more than \$500 being billed to the account before we were able to halt the process. This unexpected cost forced us to terminate further experimentation and contact Google support for clarification. The incident was also reported to our supervisor, Dr. Kim, given the financial impact and the disruption it caused to the planned experimentation schedule.

9. REFERENCES

- [1] Usenix, https://www.usenix.org/system/files/sec20fall_oest_prepub.pdf (accessed Oct. 25, 2025).
- [2] Tranco, "Tranco: A Research-Oriented Top Sites Ranking," Tranco. [Online]. Available: <https://tranco-list.eu/>. [Accessed: Oct. 26, 2025].
- [3] "chrome.webNavigation | API," *Chrome for Developers*. <https://developer.chrome.com/docs/extensions/reference/api/webNavigation>
- [4] URLhaus, "URLhaus: Sharing malicious URLs for research and analysis," Abuse.ch. [Online]. Available: <https://urlhaus.abuse.ch/>. [Accessed: Oct. 26, 2025].
- [5] VirusTotal, "VirusTotal: Analyze suspicious files, domains, IPs, and URLs," VirusTotal. [Online]. Available: <https://www.virustotal.com/>. [Accessed: Oct. 26, 2025].
- [6] R. Jabir, J. Le, and C. Nguyen, "Phishing attacks in the age of generative artificial intelligence: A systematic review of human factors," *AI*, vol. 6, no. 8, p. 174, 2025, doi: 10.3390/ai6080174. [Online]. Available: <https://doi.org/10.3390/ai6080174>
- [7] R. Wang, "Joint Semantic Detection and dissemination control of phishing attacks on social media via Llama-based modeling," Google, https://scholar.google.com/citations?view_op=view_citation&hl=en&user=flf1oaMAAAAJ&citation_for_view=flf1oaMAAAAJ%3AmaZDTaKrznsC (accessed Oct. 25, 2025).
- [8] N. S. Gurjar, S. S. S. R, C. S. Kumar, and K. K. S, "WebSecAsst - A machine learning based Chrome extension | IEEE conference publication | IEEE Xplore," IEEE Xplore, <https://ieeexplore.ieee.org/document/9488953/> (accessed Oct. 26, 2025).
- [9] Darktrace, "Darktrace: Cyber AI for Enterprise Security," Darktrace. [Online]. Available: <https://www.darktrace.com/>. [Accessed: Oct. 26, 2025].
- [10] PIXM, "PIXM Security: AI-Powered Phishing Protection," PIXM Security. [Online]. Available: <https://pixmapsecurity.com/>. [Accessed: Oct. 26, 2025].
- [11] Google Cloud, "Generative AI Studio," Google Cloud Platform. [Online]. Available: <https://cloud.google.com/generative-ai-studio>. [Accessed: Oct. 26, 2025].
- [12] "chrome.identity," *Chrome for Developers*, 2025. <https://developer.chrome.com/docs/extensions/reference/api/identity> (accessed Dec. 03, 2025).
- [12] C. E. Shannon, "A mathematical theory of communication," in *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, July 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x. (accessed Dec. 03, 2025).
- [13] Levenshtein, V. (1966). Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics-Doklady*, 10, 707--710. (accessed Dec. 03, 2025).

Appendix



Figure 1. The Training Accuracy of Gemini-2.5-Lite Model



Figure 2. The Training Loss of Gemini-2.5-Lite Model



Figure 3. The Training Accuracy of Gemini-2.5-Flash Model



Figure 4. The Training Loss of Gemini-2.5-Flash Model



Figure 5. The Training Accuracy of Gemini-2.5-Pro Model

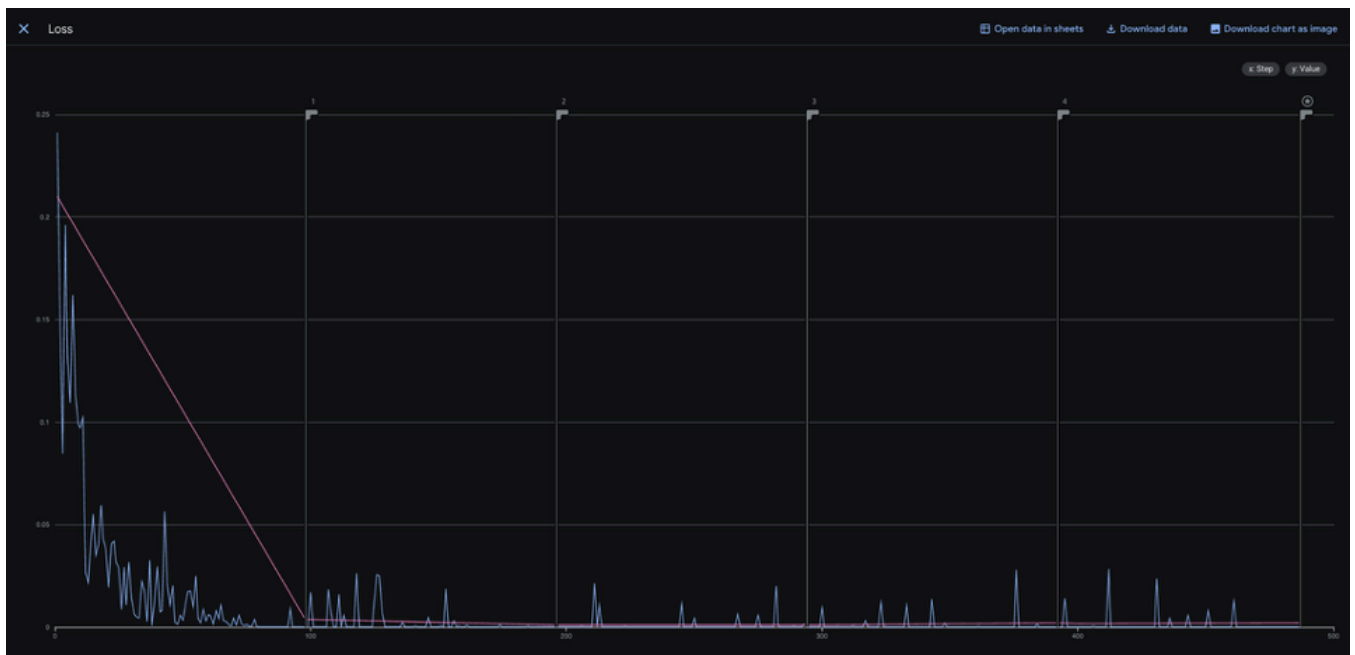


Figure 6. The Training Loss of Gemini-2.5-Pro Model